```
================================================================================
  MONGO ARCHIVER — Spring Boot (v3)
  Configurable cross-collection MongoDB archiver with cron scheduling
  and automatic temporary index management
================================================================================


================================================================================
  PROJECT STRUCTURE
================================================================================

  mongo-archiver-java-v3/
  ├── pom.xml
  ├── README.md
  └── src/main/
      ├── resources/
      │   └── application.yml                        ← Only file you edit for new
entities
      └── java/com/archiver/
          ├── MongoArchiverApplication.java           ← Entry point
          ├── config/
          │   └── ArchiverProperties.java             ← @ConfigurationProperties
binding
          ├── core/
          │   └── Archiver.java                       ← Core archive engine
          ├── job/
          │   └── ArchiveCronJob.java                 ← @Scheduled cron trigger
          ├── model/
          │   └── ArchiveLogEntry.java                ← archiveLog document model
          └── util/
              ├── ArchiveLogger.java                  ← Writes outcomes to
archiveLog
              ├── IdPathResolver.java                 ← Dot-notation query builder
              └── IndexManager.java                   ← Temporary index lifecycle
(NEW)


================================================================================
  HOW IT WORKS
================================================================================

  1. ANCHOR QUERY
     The anchor collection is queried for records where createdTsField < now -
archiveAfterDays.
```

Only the anchor uses time as the archival gate. Related collections are never filtered by time — they are looked up purely by entity ID.

2. TEMPORARY INDEX MANAGEMENT (IndexManager)
Before any batching begins, IndexManager ensures an index exists on every field that will be queried:

- Anchor collection : createdTsField (cutoff query)
- Anchor collection : idField (per-entity archive query)
- Each related coll : idPath (per-entity query)

For each field:

┌ Index already exists? ⟶ Reuse it. Record as PRE-EXISTING. Never drop.
└ No index exists? ⟶ Create one named "_arch_tmp_<field>".
Record as OWNED. Drop in finally block.

Cleanup is in a finally block in Archiver.run() — temporary indexes are ALWAYS dropped even if the run crashes mid-way.

3. BATCH PROCESSING
Matched entity IDs are split into chunks of batchSize to cap memory usage.

4. PER ENTITY — related collections first, anchor last
For each entity ID:
a. All related collections are archived (find → bulk insert → optionally delete).
b. Only if EVERY related collection succeeded → anchor is archived and deleted.
c. If any related collection failed → anchor is LEFT UNTOUCHED.
The next createdTs sweep will naturally retry this entity.

5. DUPLICATE SAFETY
BulkOperations.UNORDERED is used for inserts. Duplicate _id errors on re-runs are caught and logged — the run is not aborted.

6. AUDIT LOG
Every outcome (SUCCESS / FAILED / NO_DOCUMENTS) is upserted into the archiveLog collection keyed on (jobName, entityId, collection).

==========

TEMPORARY INDEX NAMING

==========

Indexes created by this tool are named with the prefix: _arch_tmp_

Example names:

```
    _arch_tmp_createdAt          on anchor.createdAt
    _arch_tmp_orderId            on anchor.orderId
    _arch_tmp_order_id           on orderPayments.order.id  (dots → underscores)
    _arch_tmp_meta_referenceId   on orderEvents.meta.referenceId

  If a run crashes before cleanup, you can manually drop leftover indexes with:
    db.<collection>.getIndexes()
    db.<collection>.dropIndex("_arch_tmp_<fieldname>")


================================================================================
  CONFIGURATION REFERENCE
================================================================================

  Environment variables:

  ┌──────────────────────┬──────────────────────────────────┬──────────────────────┐
  │ Variable             │ Default                          │ Purpose              │
  ├──────────────────────┼──────────────────────────────────┼──────────────────────┤
  │ MONGO_URI            │ mongodb://localhost:27017        │ MongoDB URI          │
  │ MONGO_DB             │ appdb                            │ Database name        │
  └──────────────────────┴──────────────────────────────────┴──────────────────────┘

  Common cron expressions (Spring 6-field format):
    0 0 2 * * ?      Every day at 2:00 AM
    0 0 */6 * * ?    Every 6 hours
    0 0 2 ? * MON    Every Monday at 2:00 AM
    0 0 2 1 * ?      First of every month at 2:00 AM

  ID path examples:
    "orderId"           matches  { "orderId": "abc" }
    "order.id"          matches  { "order": { "id": "abc" } }
    "meta.referenceId"  matches  { "meta": { "referenceId": "abc" } }
    "payload.orderId"   matches  { "payload": { "orderId": "abc" } }

  Adding a new entity type — edit application.yml only:
    - name: invoices
      batch-size: 50
      anchor:
        collection: invoices
        id-field: invoiceId
        created-ts-field: issuedAt
        archive-after-days: 180
        delete-after-archive: true
        archive-collection: invoices_archive
      related:
        - collection: invoiceLineItems
          id-path: invoiceId
```

```
          delete-after-archive: true
          archive-collection: invoiceLineItems_archive


================================================================================
  RUNNING
================================================================================

  # Build
  mvn clean package -DskipTests

  # Scheduled mode
  java -jar target/mongo-archiver-1.0.0.jar

  # One-shot / backfill / testing
  java -jar target/mongo-archiver-1.0.0.jar --run-now

  # With custom connection
  MONGO_URI=mongodb://prod-host:27017 MONGO_DB=mydb java -jar target/mongo-
archiver-1.0.0.jar


================================================================================
  MONITORING
================================================================================

  Query the archiveLog collection:

  // All failures
  db.archiveLog.find({ status: "FAILED" }).sort({ runAt: -1 })

  // Failures for a specific job
  db.archiveLog.find({ jobName: "orders", status: "FAILED" })

  // Full history for one entity
  db.archiveLog.find({ entityId: "abc123" }).sort({ runAt: -1 })

  // Summary by job + status
  db.archiveLog.aggregate([
    { $group: { _id: { job: "$jobName", status: "$status" }, count: { $sum: 1 } }
}
  ])

  // Check for any leftover temporary indexes after a crash
  db.orders.getIndexes().filter(i => i.name.startsWith("_arch_tmp_"))
```

```
================================================================================
  FILE: pom.xml
================================================================================

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.3</version>
        <relativePath/>
    </parent>

    <groupId>com.archiver</groupId>
    <artifactId>mongo-archiver</artifactId>
    <version>1.0.0</version>
    <name>mongo-archiver</name>
    <description>Configurable cross-collection MongoDB archiver</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Spring Boot core (scheduling, config binding, logging) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>

        <!-- Spring Data MongoDB — provides MongoTemplate + MongoDatabaseFactory -
->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-mongodb</artifactId>
        </dependency>

        <!-- Lombok — reduces boilerplate on config model classes -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
```

```
        </dependency>

        <!-- Generates IDE metadata for @ConfigurationProperties classes -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-configuration-processor</artifactId>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>


================================================================================
  FILE: src/main/resources/application.yml
================================================================================

spring:
  data:
    mongodb:
      uri: ${MONGO_URI:mongodb://localhost:27017}
      database: ${MONGO_DB:appdb}

archiver:
```

```
  # Spring cron expression (6 fields: second minute hour day month weekday)
  # Default: every day at 2:00 AM
  schedule: "0 0 2 * * ?"
  timezone: UTC

  # Collection where archive run results are persisted for auditing
  archive-log-collection: archiveLog

  # ── Archive jobs ──────────────────────────────────────────────────────
  # Each job = one logical entity type (e.g. orders, invoices, sessions).
  # Add new entity types here — no Java changes needed.
  jobs:

    - name: orders
      batch-size: 100

      # The anchor drives WHAT to archive and WHEN (via createdTs age).
      anchor:
        collection: orders
        id-field: orderId            # Top-level field on the anchor collection
        created-ts-field: createdAt  # Timestamp used to decide if record is old
enough
        archive-after-days: 90
        delete-after-archive: true
        archive-collection: orders_archive

      # All other collections that share the same entity ID.
      # Each can declare a different field name/path for the ID.
      related:
        - collection: orderItems
          id-path: orderId           # Top-level
          delete-after-archive: true
          archive-collection: orderItems_archive

        - collection: orderPayments
          id-path: order.id          # Nested — dot notation supported
          delete-after-archive: true
          archive-collection: orderPayments_archive

        - collection: orderEvents
          id-path: meta.referenceId  # Deeply nested
          delete-after-archive: false  # Copy only — keep source
          archive-collection: orderEvents_archive

        - collection: orderNotifications
          id-path: payload.orderId
          delete-after-archive: true
```

```
          archive-collection: orderNotifications_archive

    # ── Add more entity types below ──────────────────────────────
    # - name: invoices
    #   batch-size: 50
    #   anchor:
    #     collection: invoices
    #     id-field: invoiceId
    #     created-ts-field: issuedAt
    #     archive-after-days: 180
    #     delete-after-archive: true
    #     archive-collection: invoices_archive
    #   related:
    #     - collection: invoiceLineItems
    #       id-path: invoiceId
    #       delete-after-archive: true
    #       archive-collection: invoiceLineItems_archive

logging:
  level:
    com.archiver: INFO
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} [%-5level] %logger{36} - %msg%n"
    file: "%d{yyyy-MM-dd HH:mm:ss} [%-5level] %logger{36} - %msg%n"
  file:
    name: logs/archiver.log
    max-size: 10MB
    max-history: 5


================================================================================
  FILE: src/main/java/com/archiver/MongoArchiverApplication.java
================================================================================

package com.archiver;

import com.archiver.job.ArchiveCronJob;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.scheduling.annotation.EnableScheduling;

/**
```

```
 * MongoArchiverApplication
 *
 * <p>Entry point for the MongoDB archive service.
 *
 * <p>Modes:
 * <ul>
 *   <li><b>Scheduled (default)</b> — runs on the cron expression in
application.yml.</li>
 *   <li><b>One-shot</b> — pass {@code --run-now} as a program argument to execute
 *       immediately and exit. Useful for backfills or CI testing.</li>
 * </ul>
 *
 * <p>Run:
 * <pre>
 *   # Scheduled mode
 *   java -jar mongo-archiver.jar
 *
 *   # One-shot / backfill
 *   java -jar mongo-archiver.jar --run-now
 *
 *   # With overridden connection strings
 *   MONGO_PRIMARY_URI=mongodb://prod-host:27017 java -jar mongo-archiver.jar
 * </pre>
 */
@Slf4j
@SpringBootApplication
@EnableScheduling
@RequiredArgsConstructor
public class MongoArchiverApplication implements ApplicationRunner {

    private final ArchiveCronJob archiveCronJob;

    public static void main(String[] args) {
        SpringApplication.run(MongoArchiverApplication.class, args);
    }

    /**
     * After full context startup: if {@code --run-now} was passed, execute
immediately and exit.
     */
    @Override
    public void run(ApplicationArguments args) {
        if (args.containsOption("run-now")) {
            log.info("[MongoArchiverApplication] --run-now flag detected;
executing immediately");
            archiveCronJob.runNow();
            log.info("[MongoArchiverApplication] One-shot run complete — shutting
```

```
down");
            System.exit(0);
        }

        log.info("[MongoArchiverApplication] Running in scheduled mode");
    }
}
```

```
================================================================================
  FILE: src/main/java/com/archiver/config/ArchiverProperties.java
================================================================================

package com.archiver.config;

import lombok.Data;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

import java.util.List;

/**
 * Binds the entire {@code archiver.*} block from application.yml.
 *
 * Single-DB version — no connection registry, no per-collection connection names.
 * Spring Boot auto-configures MongoTemplate from spring.data.mongodb.*
properties.
 */
@Data
@Component
@ConfigurationProperties(prefix = "archiver")
public class ArchiverProperties {

    /** Spring cron expression (6 fields). Default: 2 AM daily. */
    private String schedule = "0 0 2 * * ?";

    /** Timezone for the cron schedule. */
    private String timezone = "UTC";

    /** Collection name for archive run logs. */
    private String archiveLogCollection = "archiveLog";

    /** One entry per logical entity type to archive (orders, invoices, etc.). */
    private List<JobProperties> jobs;

    // ── Nested config classes ──────────────────────────────────────────
```

```
    @Data
    public static class JobProperties {
        /** Human-readable name used in logs and archiveLog entries. */
        private String name;

        /** Number of entity IDs to process per cron tick. */
        private int batchSize = 100;

        /** The collection that drives the "old enough to archive?" decision. */
        private AnchorProperties anchor;

        /** All other collections that share the same entity ID. */
        private List<RelatedProperties> related;
    }

    @Data
    public static class AnchorProperties {
        private String collection;

        /** Top-level field holding the entity ID in the anchor collection. */
        private String idField;

        /** Timestamp field used to measure record age. */
        private String createdTsField;

        /** Archive records older than this many days. */
        private int archiveAfterDays;

        /** Remove the source document after a successful archive write. */
        private boolean deleteAfterArchive = true;

        /** Name of the archive collection to write into (same DB). */
        private String archiveCollection;
    }

    @Data
    public static class RelatedProperties {
        private String collection;

        /**
         * Dot-notation path to the entity ID within this collection's documents.
         * Examples: {@code "orderId"}, {@code "order.id"}, {@code
"meta.referenceId"}
         */
        private String idPath;

        private boolean deleteAfterArchive = true;
```

```
        /** Name of the archive collection to write into (same DB). */
        private String archiveCollection;
    }
}


================================================================================
  FILE: src/main/java/com/archiver/core/Archiver.java
================================================================================

package com.archiver.core;

import com.archiver.config.ArchiverProperties;
import com.archiver.config.ArchiverProperties.AnchorProperties;
import com.archiver.config.ArchiverProperties.JobProperties;
import com.archiver.config.ArchiverProperties.RelatedProperties;
import com.archiver.model.ArchiveLogEntry;
import com.archiver.model.ArchiveLogEntry.Status;
import com.archiver.util.ArchiveLogger;
import com.archiver.util.IdPathResolver;
import com.archiver.util.IndexManager;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.bson.Document;
import org.springframework.data.mongodb.core.BulkOperations;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.query.Query;

import java.util.ArrayList;
import java.util.Calendar;
import java.util.Date;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/**
 * Archiver
 *
 * <p>Core archive engine for one configured job.
 *
 * <h3>Index strategy</h3>
 * <p>Before batching begins, an {@link IndexManager} ensures an index exists on
 * every queried field:
 * <ul>
 *   <li>Anchor collection: {@code createdTsField} (cutoff query) and {@code
idField}
```

```
 *       (per-entity archive query).</li>
 *   <li>Each related collection: {@code idPath} (per-entity query).</li>
 * </ul>
 * If an index already exists on a field it is reused and never dropped.
 * If a temporary index was created by this run, it is dropped in a {@code
finally}
 * block — guaranteeing cleanup even on mid-run failures.
 *
 * <h3>Anchor-last strategy</h3>
 * <p>Related collections are archived first. The anchor is archived and deleted
 * only if every related collection succeeded. On any related failure the anchor
 * stays in the source DB so the next {@code createdTs} sweep retries naturally.
 */
@Slf4j
@RequiredArgsConstructor
public class Archiver {

    private final JobProperties jobConfig;
    private final MongoTemplate mongoTemplate;
    private final ArchiveLogger archiveLogger;

    // ── Entry point ──────────────────────────────────────────────────────

    public void run() {
        String jobName = jobConfig.getName();
        Date cutoff = getCutoffDate(jobConfig.getAnchor().getArchiveAfterDays());

        log.info("[Archiver:{}] Starting — archiving records with {} < {}",
            jobName, jobConfig.getAnchor().getCreatedTsField(), cutoff);

        // One IndexManager per run — tracks only indexes created by THIS run
        IndexManager indexManager = new IndexManager(mongoTemplate);

        try {
            // ── Step 1: ensure indexes on all queried fields before any batching
──
            ensureIndexes(indexManager);

            // ── Step 2: fetch entity IDs from anchor ─────────────────────
            List<String> ids = fetchAnchorIds(cutoff);
            if (ids.isEmpty()) {
                log.info("[Archiver:{}] No records to archive", jobName);
                return;
            }

            log.info("[Archiver:{}] {} entities to process", jobName, ids.size());
```

```
            // ── Step 3: process in batches ──────────────────────────────────
            int batchSize = jobConfig.getBatchSize();
            int totalBatches = (int) Math.ceil((double) ids.size() / batchSize);

            for (int i = 0; i < ids.size(); i += batchSize) {
                List<String> batch = ids.subList(i, Math.min(i + batchSize,
ids.size()));
                log.info("[Archiver:{}] Batch {}/{} ({} IDs)",
                    jobName, (i / batchSize) + 1, totalBatches, batch.size());
                processBatch(batch);
            }

            log.info("[Archiver:{}] Archive run complete", jobName);

        } finally {
            // ── Always drop temporary indexes — even on exception / partial
failure ──
            indexManager.dropCreatedIndexes();
        }
    }

    // ─── Index management ──────────────────────────────────────────────────

    /**
     * Ensure an index exists on every field this job will query.
     * Pre-existing indexes are noted but never dropped.
     * Newly created indexes are owned by this run and dropped in the finally
block.
     */
    private void ensureIndexes(IndexManager indexManager) {
        AnchorProperties anchor = jobConfig.getAnchor();

        // Anchor — two fields: the timestamp gate and the ID lookup
        indexManager.ensureIndex(anchor.getCollection(),
anchor.getCreatedTsField());
        indexManager.ensureIndex(anchor.getCollection(), anchor.getIdField());

        // Each related collection — the ID path used for per-entity queries
        for (RelatedProperties rel : jobConfig.getRelated()) {
            indexManager.ensureIndex(rel.getCollection(), rel.getIdPath());
        }

        log.info("[Archiver:{}] Index check complete — {} temporary index(es)
created",
            jobConfig.getName(), indexManager.ownedCount());
    }
```

```
    // ── Batch processing ──────────────────────────────────────────────

    private void processBatch(List<String> entityIds) {
        for (String entityId : entityIds) {
            boolean allRelatedSucceeded = archiveRelatedCollections(entityId);

            if (allRelatedSucceeded) {
                archiveAnchor(entityId);
            } else {
                log.warn("[Archiver:{}] Skipping anchor for entity '{}' — " +
                    "related collection failure. Will retry on next createdTs
sweep.",
                    jobConfig.getName(), entityId);
            }
        }
    }

    private boolean archiveRelatedCollections(String entityId) {
        boolean allSucceeded = true;
        for (RelatedProperties rel : jobConfig.getRelated()) {
            boolean ok = archiveCollection(
                entityId,
                rel.getCollection(),
                rel.getIdPath(),
                rel.getArchiveCollection(),
                rel.isDeleteAfterArchive()
            );
            if (!ok) allSucceeded = false;
        }
        return allSucceeded;
    }

    private void archiveAnchor(String entityId) {
        AnchorProperties anchor = jobConfig.getAnchor();
        archiveCollection(
            entityId,
            anchor.getCollection(),
            anchor.getIdField(),
            anchor.getArchiveCollection(),
            anchor.isDeleteAfterArchive()
        );
    }

    // ── Core archive operation ────────────────────────────────────────

    /**
     * Find all documents for {@code entityId} in {@code srcCollection},
```

```
     * write them to {@code archiveCollection}, optionally delete from source.
     *
     * @return true on success or no-documents; false on error
     */
    private boolean archiveCollection(
        String entityId,
        String srcCollection,
        String idPath,
        String archiveCollection,
        boolean deleteAfterArchive
    ) {
        String jobName = jobConfig.getName();
        try {
            Query query = IdPathResolver.buildSingleQuery(idPath, entityId);
            List<Document> docs = mongoTemplate.find(query, Document.class,
srcCollection);

            if (docs.isEmpty()) {
                log.debug("[Archiver:{}] No documents in '{}' for entity '{}'",
                    jobName, srcCollection, entityId);
                writeLog(jobName, entityId, srcCollection, Status.NO_DOCUMENTS, 0,
null);
                return true;
            }

            // Stamp with archive metadata
            Date archivedAt = new Date();
            docs.forEach(doc -> {
                doc.append("_archivedAt", archivedAt);
                doc.append("_archivedFrom", srcCollection);
            });

            // Bulk insert — UNORDERED tolerates duplicate _id on re-runs
            int inserted = bulkInsert(docs, archiveCollection, jobName, entityId,
srcCollection);

            if (deleteAfterArchive) {
                mongoTemplate.remove(query, srcCollection);
            }

            log.info("[Archiver:{}] Archived {} doc(s): '{}' → '{}' (entity:
'{}')",
                jobName, docs.size(), srcCollection, archiveCollection, entityId);

            writeLog(jobName, entityId, srcCollection, Status.SUCCESS,
docs.size(), null);
            return true;
```

```
        } catch (Exception e) {
            log.error("[Archiver:{}] FAILED '{}' for entity '{}': {}",
                jobName, srcCollection, entityId, e.getMessage(), e);
            writeLog(jobName, entityId, srcCollection, Status.FAILED, 0,
e.getMessage());
            return false;
        }
    }

    // ── Bulk insert with duplicate tolerance ──────────────────────────

    private int bulkInsert(
        List<Document> docs, String collection,
        String jobName, String entityId, String srcCollection
    ) {
        try {
            BulkOperations bulk =
mongoTemplate.bulkOps(BulkOperations.BulkMode.UNORDERED, collection);
            docs.forEach(bulk::insert);
            return bulk.execute().getRawResult().getInsertedCount();

        } catch (org.springframework.data.mongodb.BulkOperationException e) {
            int inserted = e.getResult() != null ?
e.getResult().getInsertedCount() : 0;
            int dupes = docs.size() - inserted;
            if (dupes > 0) {
                log.warn("[Archiver:{}] '{}' entity='{}' — {} already archived, {}
newly inserted",
                    jobName, srcCollection, entityId, dupes, inserted);
            }
            return inserted;
        }
    }

    // ── Anchor ID fetching ────────────────────────────────────────────

    private List<String> fetchAnchorIds(Date cutoff) {
        AnchorProperties anchor = jobConfig.getAnchor();
        try {
            Query query =
IdPathResolver.buildCutoffQuery(anchor.getCreatedTsField(), cutoff);
            query.fields().include(anchor.getIdField()).exclude("_id");

            Set<String> ids = new LinkedHashSet<>();
            mongoTemplate.find(query, Document.class, anchor.getCollection())
                .forEach(doc -> {
```

```
                    String id = IdPathResolver.extractIdAsString(doc,
anchor.getIdField());
                    if (id != null) ids.add(id);
                });

            log.info("[Archiver:{}] Found {} anchor IDs in '{}'",
                jobConfig.getName(), ids.size(), anchor.getCollection());
            return new ArrayList<>(ids);

        } catch (Exception e) {
            log.error("[Archiver:{}] Failed to fetch anchor IDs from '{}': {}",
                jobConfig.getName(), anchor.getCollection(), e.getMessage(), e);
            return List.of();
        }
    }

    // ── Helpers ──────────────────────────────────────────────────────────

    private Date getCutoffDate(int days) {
        Calendar cal = Calendar.getInstance();
        cal.add(Calendar.DATE, -days);
        return cal.getTime();
    }

    private void writeLog(String jobName, String entityId, String collection,
                          Status status, int count, String error) {
        archiveLogger.log(ArchiveLogEntry.builder()
            .jobName(jobName)
            .entityId(entityId)
            .collection(collection)
            .status(status)
            .archivedCount(count)
            .errorMessage(error)
            .runAt(new Date())
            .build());
    }
}
```

```
================================================================================
  FILE: src/main/java/com/archiver/job/ArchiveCronJob.java
================================================================================

package com.archiver.job;

import com.archiver.config.ArchiverProperties;
import com.archiver.core.Archiver;
```

```
import com.archiver.util.ArchiveLogger;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.time.Instant;
import java.util.concurrent.atomic.AtomicBoolean;

/**
 * ArchiveCronJob
 *
 * Triggers the archive run on schedule. Delegates to one {@link Archiver}
 * instance per configured job. All jobs share the same auto-configured
 * {@link MongoTemplate} (single DB).
 */
@Slf4j
@Component
@RequiredArgsConstructor
public class ArchiveCronJob {

    private final ArchiverProperties properties;
    private final MongoTemplate mongoTemplate;
    private final ArchiveLogger archiveLogger;

    /** Prevents overlapping runs if a job takes longer than the cron interval. */
    private final AtomicBoolean running = new AtomicBoolean(false);

    @PostConstruct
    public void init() {
        archiveLogger.ensureIndexes();
        log.info("[ArchiveCronJob] Initialised. Schedule='{}' timezone='{}'",
            properties.getSchedule(), properties.getTimezone());
        log.info("[ArchiveCronJob] {} job(s) configured: {}",
            properties.getJobs().size(),

properties.getJobs().stream().map(ArchiverProperties.JobProperties::getName).toList
    }

    @Scheduled(cron = "${archiver.schedule}", zone = "${archiver.timezone}")
    public void runScheduled() {
        execute("scheduled");
    }

    /** Programmatic trigger — useful for one-shot runs or testing. */
```

```
    public void runNow() {
        execute("manual");
    }

    // ── Private ──────────────────────────────────────────────────────────

    private void execute(String trigger) {
        if (!running.compareAndSet(false, true)) {
            log.warn("[ArchiveCronJob] Previous run still in progress — skipping
{} trigger", trigger);
            return;
        }

        long start = System.currentTimeMillis();
        log.info("[ArchiveCronJob] Run started at {} (trigger: {})",
Instant.now(), trigger);

        try {
            for (ArchiverProperties.JobProperties jobConfig :
properties.getJobs()) {
                try {
                    new Archiver(jobConfig, mongoTemplate, archiveLogger).run();
                } catch (Exception e) {
                    log.error("[ArchiveCronJob] Unhandled error in job '{}': {}",
                        jobConfig.getName(), e.getMessage(), e);
                }
            }
        } finally {
            running.set(false);
            log.info("[ArchiveCronJob] Completed in {}s",
                String.format("%.1f", (System.currentTimeMillis() - start) /
1000.0));
        }
    }
}
```

```
================================================================================
  FILE: src/main/java/com/archiver/model/ArchiveLogEntry.java
================================================================================

package com.archiver.model;

import lombok.Builder;
import lombok.Data;

import java.util.Date;
```

```
/**
 * One row in the {@code archiveLog} collection.
 * Keyed on (jobName, entityId, collection) — upserted on every run.
 *
 * {
 *   jobName      : "orders",
 *   entityId     : "abc123",
 *   collection   : "orderPayments",
 *   status       : "SUCCESS" | "FAILED" | "NO_DOCUMENTS",
 *   archivedCount: 5,
 *   errorMessage : null,
 *   runAt        : ISODate("..."),
 *   retryCount   : 2
 * }
 */
@Data
@Builder
public class ArchiveLogEntry {

    public enum Status {
        SUCCESS,
        FAILED,
        NO_DOCUMENTS
    }

    private String jobName;
    private String entityId;
    private String collection;
    private Status status;
    private int archivedCount;
    private String errorMessage;
    private Date runAt;
}
```

```
================================================================================
  FILE: src/main/java/com/archiver/util/ArchiveLogger.java
================================================================================

package com.archiver.util;

import com.archiver.config.ArchiverProperties;
import com.archiver.model.ArchiveLogEntry;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.bson.Document;
```

```
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.index.CompoundIndexDefinition;
import org.springframework.data.mongodb.core.index.Index;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;
import org.springframework.data.mongodb.core.query.Update;
import org.springframework.stereotype.Component;

import java.util.Date;

/**
 * ArchiveLogger
 *
 * Persists the outcome of each archive operation to the {@code archiveLog}
 * collection for auditing. Uses Spring Boot's auto-configured {@link
MongoTemplate}.
 *
 * Upserts keyed on (jobName, entityId, collection) so re-runs overwrite cleanly.
 */
@Slf4j
@Component
@RequiredArgsConstructor
public class ArchiveLogger {

    private final MongoTemplate mongoTemplate;
    private final ArchiverProperties properties;

    /**
     * Upsert a log entry for one (entity, collection) outcome.
     * Silently swallows exceptions so logging never crashes the archive job.
     */
    public void log(ArchiveLogEntry entry) {
        try {
            Query query = Query.query(
                Criteria.where("jobName").is(entry.getJobName())
                    .and("entityId").is(entry.getEntityId())
                    .and("collection").is(entry.getCollection())
            );

            Update update = new Update()
                .set("status", entry.getStatus().name())
                .set("archivedCount", entry.getArchivedCount())
                .set("errorMessage", entry.getErrorMessage())
                .set("runAt", new Date())
                .inc("retryCount", 1)
                .setOnInsert("createdAt", new Date());
```

```
            mongoTemplate.upsert(query, update,
properties.getArchiveLogCollection());

        } catch (Exception e) {
            log.warn("[ArchiveLogger] Failed to write log for entity={}
collection={}: {}",
                entry.getEntityId(), entry.getCollection(), e.getMessage());
        }
    }

    /**
     * Ensure indexes on the archiveLog collection. Idempotent — safe to call on
every startup.
     */
    public void ensureIndexes() {
        try {
            String col = properties.getArchiveLogCollection();

            mongoTemplate.indexOps(col).ensureIndex(
                new CompoundIndexDefinition(
                    new Document("jobName", 1).append("entityId",
1).append("collection", 1)
                ).unique()
            );
            mongoTemplate.indexOps(col).ensureIndex(
                new CompoundIndexDefinition(new Document("status",
1).append("jobName", 1))
            );
            mongoTemplate.indexOps(col).ensureIndex(
                new Index("runAt", Sort.Direction.DESC)
            );

            log.info("[ArchiveLogger] Indexes ensured on '{}'", col);
        } catch (Exception e) {
            log.warn("[ArchiveLogger] Index creation warning: {}",
e.getMessage());
        }
    }
}
```

```
================================================================================
  FILE: src/main/java/com/archiver/util/IdPathResolver.java
================================================================================
```

```
package com.archiver.util;
```

```
import org.bson.Document;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;

import java.util.Date;
import java.util.List;

/**
 * IdPathResolver
 *
 * <p>Builds Spring Data MongoDB {@link Query} objects from dot-notation ID paths,
 * and walks in-memory {@link Document} objects to extract values at those paths.
 *
 * <p>MongoDB's query engine natively supports dot-notation in field names, so
 * {@code Criteria.where("meta.referenceId")} correctly matches nested documents.
 *
 * <p>Examples:
 * <pre>
 *   resolveValue(doc, "orderId")          → "abc123"
 *   resolveValue(doc, "order.id")         → "abc123"
 *   resolveValue(doc, "meta.referenceId") → "abc123"
 *
 *   buildSingleQuery("order.id", "abc")   → Query: { "order.id": "abc" }
 *   buildBatchQuery("order.id", ids)      → Query: { "order.id": { $in: [...] } }
 *   buildCutoffQuery("createdAt", date)   → Query: { "createdAt": { $lt: date } }
 * </pre>
 */
public final class IdPathResolver {

    private IdPathResolver() {}

    /**
     * Walk a dot-notation path on a {@link Document}, returning the nested value.
     *
     * @param doc  source document
     * @param path dot-notation path, e.g. {@code "meta.referenceId"}
     * @return the resolved value, or {@code null} if any segment is missing
     */
    public static Object resolveValue(Document doc, String path) {
        String[] parts = path.split("\\.");
        Object current = doc;
        for (String part : parts) {
            if (!(current instanceof Document nested)) {
                return null;
            }
            current = nested.get(part);
```

```
        }
        return current;
    }

    /**
     * Build a Spring Data {@link Query} matching a single entity ID at the given
path.
     *
     * @param idPath   dot-notation field path, e.g. {@code "order.id"}
     * @param idValue  the ID value to match
     * @return {@link Query}
     */
    public static Query buildSingleQuery(String idPath, Object idValue) {
        return Query.query(Criteria.where(idPath).is(idValue));
    }

    /**
     * Build a Spring Data {@link Query} matching a batch of entity IDs via {@code
$in}.
     *
     * @param idPath   dot-notation field path, e.g. {@code "order.id"}
     * @param idValues list of ID values
     * @return {@link Query} using {@code $in}
     */
    public static Query buildBatchQuery(String idPath, List<?> idValues) {
        return Query.query(Criteria.where(idPath).in(idValues));
    }

    /**
     * Build a Spring Data {@link Query} for records where a date field is before
     * the given cutoff. Used to query the anchor collection by {@code createdTs}.
     *
     * @param tsField timestamp field name (top-level)
     * @param cutoff  upper bound (exclusive)
     * @return {@link Query}
     */
    public static Query buildCutoffQuery(String tsField, Date cutoff) {
        return Query.query(Criteria.where(tsField).lt(cutoff));
    }

    /**
     * Extract the value of a top-level field from a document as a String.
     * Returns {@code null} if the field is absent or null.
     *
     * @param doc   source document
     * @param field top-level field name
     * @return string representation, or {@code null}
```

```
     */
    public static String extractIdAsString(Document doc, String field) {
        Object value = doc.get(field);
        return value != null ? value.toString() : null;
    }
}
```

```
================================================================================
  FILE: src/main/java/com/archiver/util/IndexManager.java
================================================================================

package com.archiver.util;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.index.IndexInfo;
import org.springframework.data.mongodb.core.index.IndexOperations;
import
org.springframework.data.mongodb.core.index.MongoPersistentEntityIndexResolver;
import org.springframework.data.mongodb.core.index.PartialIndexFilter;
import org.springframework.data.mongodb.core.index.Index;

import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/**
 * IndexManager
 *
 * <p>Manages temporary indexes created for the duration of one archive run.
 *
 * <p>Behaviour per field:
 * <ul>
 *   <li>If an index on the field already exists → record it as <b>pre-
existing</b>,
 *       use it as-is, and <b>never drop it</b> after the run.</li>
 *   <li>If no index exists → create one, record it as <b>owned</b>,
 *       and drop it in {@link #dropCreatedIndexes()} when the run finishes.</li>
 * </ul>
 *
 * <p>{@link #dropCreatedIndexes()} must be called in a {@code finally} block so
 * that temporary indexes are always cleaned up even on mid-run failures.
 *
 * <p>One {@code IndexManager} instance per archive run — not a Spring singleton.
 */
```

```
@Slf4j
@RequiredArgsConstructor
public class IndexManager {

    /**
     * Prefix applied to every index name this manager creates.
     * Makes temporary indexes easy to identify if a run crashes before cleanup.
     */
    private static final String INDEX_PREFIX = "_arch_tmp_";

    private final MongoTemplate mongoTemplate;

    /**
     * Tracks indexes this manager created (as "collection::indexName" keys).
     * Pre-existing indexes are never added here and are never dropped.
     */
    private final Set<String> ownedIndexes = new LinkedHashSet<>();

    // ── Public API ──────────────────────────────────────────────────────

    /**
     * Ensure an index exists on {@code fieldPath} in {@code collection}.
     *
     * <p>If an index covering this field already exists (under any name), it is
     * reused and will not be dropped after the run. If no such index exists, a
     * temporary one is created and tracked for cleanup.
     *
     * @param collection MongoDB collection name
     * @param fieldPath  dot-notation field path, e.g. {@code "meta.referenceId"}
     */
    public void ensureIndex(String collection, String fieldPath) {
        try {
            IndexOperations indexOps = mongoTemplate.indexOps(collection);

            if (indexAlreadyExists(indexOps, fieldPath)) {
                log.info("[IndexManager] Pre-existing index found on '{}.{}' —
will not drop after run",
                    collection, fieldPath);
                return;
            }

            // No existing index — create a temporary one
            String indexName = INDEX_PREFIX + fieldPath.replace(".", "_");
            indexOps.ensureIndex(
                new Index().on(fieldPath,
org.springframework.data.domain.Sort.Direction.ASC)
                           .named(indexName)
```

```
            );

            String key = collection + "::" + indexName;
            ownedIndexes.add(key);
            log.info("[IndexManager] Created temporary index '{}' on '{}.{}'",
                indexName, collection, fieldPath);

        } catch (Exception e) {
            // Non-fatal — log and continue. Query will still work, just without
the index.
            log.warn("[IndexManager] Could not ensure index on '{}.{}': {}",
                collection, fieldPath, e.getMessage());
        }
    }

    /**
     * Drop all indexes that this manager created during the current run.
     * Pre-existing indexes are never touched.
     *
     * <p>Must be called in a {@code finally} block to guarantee cleanup on
failure.
     */
    public void dropCreatedIndexes() {
        if (ownedIndexes.isEmpty()) {
            return;
        }

        log.info("[IndexManager] Dropping {} temporary index(es)...",
ownedIndexes.size());

        for (String key : ownedIndexes) {
            String[] parts = key.split("::", 2);
            String collection = parts[0];
            String indexName  = parts[1];
            try {
                mongoTemplate.indexOps(collection).dropIndex(indexName);
                log.info("[IndexManager] Dropped temporary index '{}' on '{}'",
indexName, collection);
            } catch (Exception e) {
                log.warn("[IndexManager] Failed to drop index '{}' on '{}': {}",
                    indexName, collection, e.getMessage());
            }
        }

        ownedIndexes.clear();
    }
```

```
    /**
     * How many temporary indexes this manager has created (useful for logging).
     */
    public int ownedCount() {
        return ownedIndexes.size();
    }

    // ─── Private helpers ──────────────────────────────────────────────────

    /**
     * Check whether any existing index on the collection covers {@code fieldPath}
     * as its leading (or only) key.
     *
     * <p>MongoDB represents dot-notation fields as-is in index key documents,
     * so {@code "meta.referenceId"} in the index info will match the path
exactly.
     */
    private boolean indexAlreadyExists(IndexOperations indexOps, String fieldPath)
{
        try {
            List<IndexInfo> existing = indexOps.getIndexInfo();
            for (IndexInfo info : existing) {
                boolean covers = info.getIndexFields().stream()
                    .anyMatch(f -> f.getKey().equals(fieldPath));
                if (covers) {
                    return true;
                }
            }
        } catch (Exception e) {
            log.warn("[IndexManager] Could not read index info: {}",
e.getMessage());
        }
        return false;
    }
}


================================================================================
  END OF FILE
================================================================================
```